Reconfigurable Precision INT4-8/FP8 Digital Compute-in-Memory Macro for AI Acceleration

Jinane Bazzi[1], Mohammed E. Fouda[2,†], Ahmed Eltawil[1]
[1]King Abdullah University of Science and Technology (KAUST), Thuwal 23955, Saudi Arabia.
[2] Rain Neuromorphics, San Francisco, CA, USA. [†] Email: foudam@uci.edu

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11044064

***Abstract*—Compute-in-memory (CIM) technology has emerged as a promising solution to address the computational demands of deep neural network (DNN) models, which require substantial multiply-accumulate (MAC) operations. However, there is a growing need for reconfigurable CIM architectures that can support both integer (INT) and floating-point (FP) operations within a single design. This flexibility is crucial for optimizing efficiency and resource utilization, especially in DNN applications involving mixed-precision computations. In this work, we propose a reconfigurable precision digital macro design that accelerates MAC computations while supporting INT4, INT8, and FP8 configurations within the same architecture. Both signed and unsigned operations are supported in INT mode. To enhance performance, the proposed design uses a parallel-input approach and a mantissa parallel-alignment technique in FP mode. The macro is implemented in 40nm CMOS technology. It achieves a peak throughput of 7123.48 GOPS in INT4 mode and 1187.25 GFLOPS in FP8 mode, with peak energy efficiencies of 367.45 TOPS/W and 23.14 TFLOPS/W, respectively.**

***Index Terms*—Compute-in-memory, reconfigurable macro, multiply-accumulate, floating-point, deep neural network.**

I. Introduction

In the era of artificial intelligence (AI), deep neural networks (DNNs) have achieved remarkable success across various applications, including image recognition, natural language processing, autonomous driving, and medical diagnosis [1]–[4]. This advancement has led to the development of deeper and more complex DNN models, which require an increased number of multiply-accumulate (MAC) operations, resulting in significant computational overhead for both inference and training. Consequently, there is a growing need for specialized hardware accelerators to handle these computations efficiently.

Compute-in-memory (CIM) technology has emerged as a promising solution to the memory wall problem in conventional von Neumann architectures by integrating memory and computation to minimize data movement. CIM technology has proven to enhance computational and energy efficiency, making it ideal for the data-intensive MAC computations required by DNN models [5]. CIM architectures can be realized with different memory technologies, including SRAM [6]–[9], ReRAM [10]–[12], DRAM [13], and MRAM [14], [15]. Among these, SRAM-based macros stand out due to their fast access time, low write energy, high endurance, parallelism, and compatibility with advanced CMOS process nodes. SRAM macros can be classified into digital and analog compute types. Compared to analog-based CIM macros [6], [16]–[19], digital-based CIM (DCIM) macros [7], [8], [20]–[25] offer higher precision and are less susceptible to noise and variability, which can significantly impact the accuracy and reliability of analog computations.

Many DCIM macros have been proposed for MAC acceleration. They typically adopt a bit-serial input approach, wherein inputs are applied one bit per cycle, and multiplication results are accumulated across cycles [7], [20], [23], [25]. By contrast, applying inputs in parallel offers the advantage of lower latency and higher throughput, making it better suited for the intensive computational demands of AI workloads. Additionally, existing designs often separate floating-point (FP) and integer (INT) MAC computations, with most macros supporting only one format, either INT or FP. This limitation reduces flexibility and efficiency, particularly in applications requiring mixed-precision computations. To reduce computational and memory costs while maintaining model accuracy, DNN models are increasingly adopting low-bit precision data formats. These formats enhance efficiency by enabling more MAC calculations per clock cycle, accelerating training and inference, and reducing the memory footprint, which leads to fewer data fetches and improved overall performance [26].

To address the limitations of previous designs and meet the diverse requirements of modern DNN models, this work proposes a reconfigurable precision digital SRAM-based macro that offers flexibility by supporting both INT and FP MAC operations within the same architecture. Specifically, the macro supports low-bit INT4, INT8, and FP8 (E4M3) configurations, and in INT mode, both signed and unsigned operations are supported. In FP mode, the design utilizes a parallel-alignment mantissa technique to enhance FP performance. Additionally, the macro uses a parallel-input approach to enhance throughput, ensuring high performance across different precision requirements.

The remainder of the paper is organized as follows: II discusses the full hardware architecture implementation of the proposed reconfigurable CIM macro and its operation in both INT and FP modes. Section III presents the performance evaluation of the proposed design and compares it with previous works. Finally, Section IV concludes the paper.

II. Hardware Implementation

The overall architecture of the proposed 36Kb reconfigurable macro is illustrated in Fig. 1. The macro consists of 64 blocks, each with two 6T SRAM arrays, multipliers, adder trees, mantissa shifters, a product aligner, a post-sum unit (PSU), and a normalization and rounding unit (NRU). The macro supports two operation modes: signed and unsigned INT mode for INT4 and INT8, and FP mode for FP8 (E4M3),

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:31 UTC from IEEE Xplore. Restrictions apply.

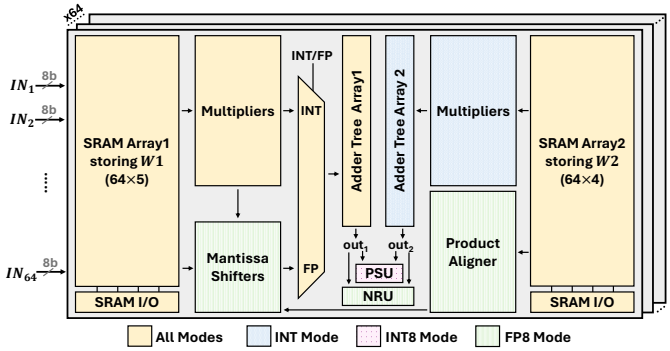


Fig. 1. Overall architecture of the proposed reconfigurable CIM macro design.

which consists of a 1-bit sign, a 4-bit exponent, and a 3-bit mantissa. Components are activated or deactivated based on the selected mode, as shown in Fig. 1. Data stored in SRAM Array1 is denoted as $W1$ and Array2 as $W2$. In INT mode, the SRAM cells store integer weight values, while in FP mode, they store weight mantissas and exponents. Inputs are applied in parallel across all macro blocks. To maximize data reuse, the SRAM arrays of the 64 blocks store different weights but receive the same 64 inputs. Each block can hold either 64×2 INT4 weights or 64 INT8 or FP8 weights. In INT4 mode, the macro performs the dot product of the input with 128 INT4 vectors of size 64 simultaneously, while in INT8 and FP8 modes, it handles the dot product with 64 vectors of size 64.

A. INT Mode (INT4/INT8)

In INT mode, the macro supports both signed and unsigned MAC operations. For signed operations, the inputs, weights, and outputs are represented in two's complement format.

1) INT4 Mode: The SRAM arrays operate independently, each storing a set of 64 4-bit weights. Each weight is stored in a row of four adjacent SRAM cells. Note that Array1 includes an extra cell in each row for FP8 mode, which is deactivated in INT mode. For the inputs, the four most significant bits (IN[7:4]) and the four least significant bits (IN[3:0]) are processed separately. Specifically, bits IN[7:4] are used for the dot product with weights stored in Array1 ($W1$), while bits IN[3:0] for Array2 ($W2$). To perform the multiplication between the inputs and weights, multiplexers are used to generate the partial products, as illustrated in Fig. 2 for Array1, with a similar implementation in Array2. Each of the four multiplexers is controlled by one bit of the 4-bit weight and outputs either a 4-bit zero or the input value, representing the partial product, based on the corresponding weight bit. Once the inputs are applied, all partial products across the rows are generated. The four partial products within each row are summed using adders to compute the product of the weight and input for that row. An adder tree is then used to sum the products across all rows to produce the final output. In this mode, each block produces two outputs, one for each array, and the computation is completed in one clock cycle.

2) INT8 Mode: The 8-bit weights are divided into two parts and stored in the SRAM arrays as follows: the four most significant bits are stored in SRAM Array1 ($W1$), while the four least significant bits are stored in SRAM Array2

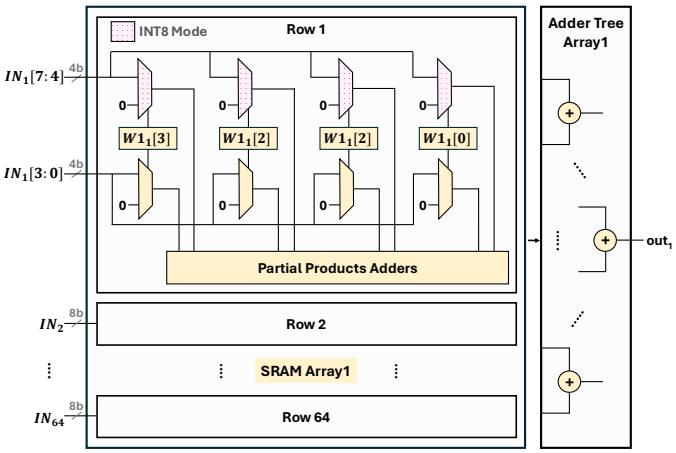


Fig. 2. Implementation of INT MAC in SRAM Array1. Note that an additional SRAM cell and multiplexer are included in each row but are not shown in the figure, as they are used only in FP mode and deactivated in INT mode.

($W2$). The least significant bits are always unsigned. The final dot product output is computed according to Eq. 1. Unlike INT4 mode, where the most and least significant bits of the inputs are processed independently, in INT8 mode, the entire 8-bit inputs (IN) are used with both weight arrays. Specifically, the 8-bit inputs are multiplied by the 4-bit weights from each array, resulting in eight partial products per row. Four additional multiplexers are used to produce these extra partial products, as shown in Fig. 2. These multiplexers are deactivated in INT4 mode using power gating. As in INT4 mode, the partial products are summed within each row and then across all rows using the adder tree. Because the same adder tree is used for both modes, some adders are deactivated in INT4 mode due to the smaller length of the partial products' sum. After the adder tree, the outputs from the two arrays are combined using the post-sum unit, which left-shifts the output of Array1 by 4 bits and adds it to the output of Array2 to obtain the final result according to Eq. 1. This process requires two clock cycles: the first cycle computes the sum of the input-weight products for each array, and the second cycle is used by the PSU to combine the results from the two arrays.

$$IN \cdot W = \sum_i IN_i(16 \times W_i^{MSB} + W_i^{LSB}) = (\text{out}_1 << 4) + \text{out}_2 \quad (1)$$

B. FP8 Mode

For FP MAC computations, a "1" bit is typically added between the sign and mantissa bits to create a fixed-point number for performing INT mantissa MAC operations. Thus, weight mantissas are stored in a format called Mantissa+, which represents {sign, 1, mantissa} in two's complement, requiring 5 bits. Consequently, an extra cell is added to each row of Array1 to accommodate this additional bit, which is deactivated in INT mode. For the inputs, the mantissas (IN[7:4]) are also applied in two's complement format, with the hidden bit signal generated based on the input's sign and mantissa bits. Bits (IN[3:0]) represent the input exponents

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:31 UTC from IEEE Xplore. Restrictions apply.

which are applied to Array2. Unlike conventional FP MAC alignment, which requires mantissa alignment before each addition, this work employs an in-memory parallel-alignment FP MAC approach. This method aligns all products after multiplication, as illustrated in Fig. 4(a), resulting in reduced area overhead and a shorter critical path. Array2 receives the 4-bit input exponent bits to compute the sum of each input exponent and weight exponent using adders, as shown in Fig. 3. To determine the appropriate shift amounts for the mantissa products, a comparison tree is used to find the maximum exponent (E_{max}). The shift amounts are then obtained by subtracting each exponent addition from E_{max}. Consequently, the exponent of the final output before normalization is E_{max}.

For MAC multiplication, the input mantissa bits are applied to Array1. The multiplication is performed similarly to INT mode (Fig. 2), with the addition of a cell controlling an extra multiplexer in each row. The main difference in this mode is the alignment of the mantissa products before they are input to the adder tree. Specifically, the mantissa product of each row, denoted as MP_i, is extended by 4 bits and right-shifted by the corresponding calculated shift amount determined by the product aligner. Shifting by 4 bits ensures high accuracy, as this typically accommodates the exponent offset between the terms effectively [9], [20].

A simple example of the operation implementation is shown in Fig. 4(b), with input and weight vectors of size 4. The product aligner in Array2 adds the exponents and identifies the maximum value among the exponent sums to calculate the shift amounts for the mantissa products. The mantissas, stored in the two's complement Mantissa+ format, are multiplied in Array1, producing the product vector (MP_1, MP_2, MP_3, MP_4). Once the shift amounts are determined, the products are right-shifted accordingly before being summed in the adder tree. Finally, a normalization and rounding unit (NRU) identifies the leading "1" in the mantissa, selects the next three bits, and adjusts the exponent to reflect the new position of the leading bit, ensuring proper normalization to the FP8 E4M3 format. By using the parallel-alignment FP MAC approach, the interaction between exponent and mantissa calculations is minimized, allowing these operations to be pipelined separately. This results in a three-stage pipeline, as shown in Fig. 4(c). Stage

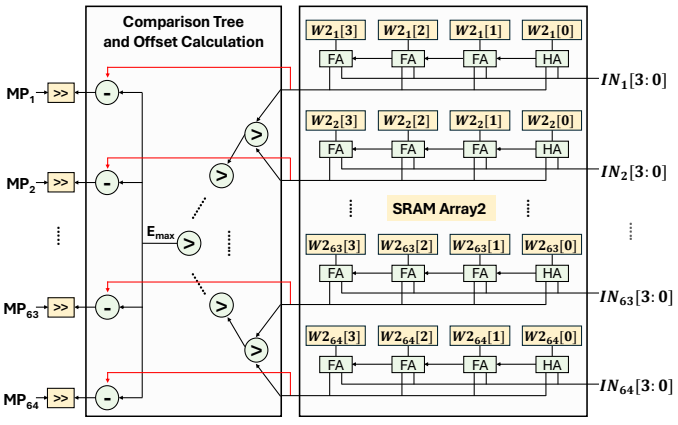


Fig. 3. Shift amounts calculation for in-memory alignment-free FP MAC.

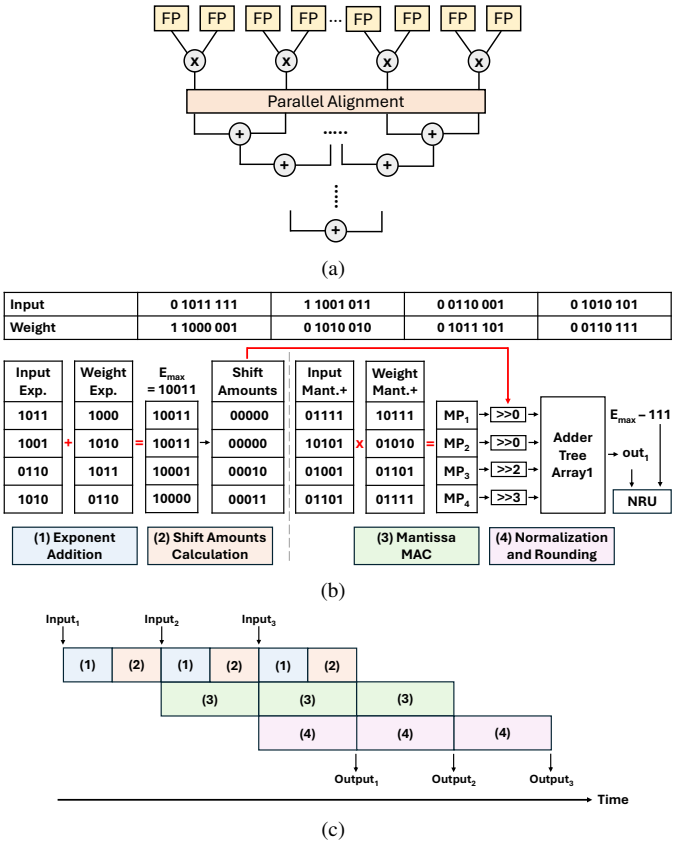


Fig. 4. (a) Parallel-alignment FP MAC approach. (b) Example implementation with input and weight vectors of size 4. (c) FP MAC pipeline.

1 involves exponent addition and shift amounts calculation, stage 2 performs mantissa MAC operations with the applied shifts for proper alignment, and stage 3 handles rounding and normalization to produce the final FP8 format.

III. Results and Discussion

The proposed macro is implemented using TSMC 40nm CMOS technology and simulated with the Cadence Spectre simulator. The simulation results show that the macro can work at 0.7 to 1V supply voltage, with a minimum clock cycle of 2.3ns when operating at V_{DD} = 1.2V, as shown in Fig. 5(a). The macro, consisting of 64 blocks each with two SRAM arrays, performs 2×64×2×64 = 16384 operations simultaneously in INT4 mode. In INT8 and FP8 modes, where the two SRAM arrays in each block are combined, it performs 2×64×64 = 8192 operations simultaneously. Fig. 5(b) shows the throughput versus V_{DD}, with the macro achieving peak throughputs of 7123.48 GOPS in INT4 mode, 1780.87 GOPS in INT8 mode, and 1187.25 GFLOPS in FP8 mode at V_{DD} = 1.2V. Fig. 5(c)-(e) shows the energy efficiency for different V_{DD} values and toggle rates for input and weight data. Energy efficiency is optimized at the lowest supply voltage of 0.7V and the lowest toggle rate of 10%. The macro achieves peak energy efficiencies of 367.45 TOPS/W in INT4 mode, 77.25 TOPS/W in INT8 mode, and 23.14 TFLOPS/W in FP8 mode.

Table I compares the proposed design with state-of-the-art (SOTA) digital SRAM-based CIM macros. The proposed

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:31 UTC from IEEE Xplore. Restrictions apply.

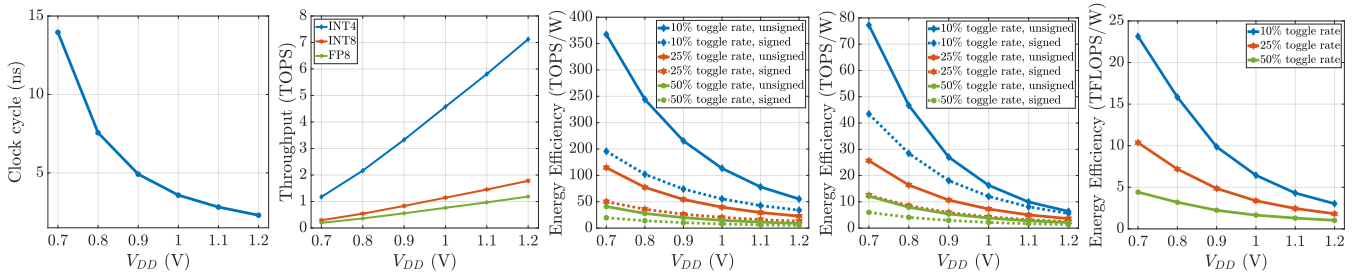


Fig. 5. (a) Clock cycle, (b) throughput, and (c)-(e) energy efficiency vs V_{DD} for INT4, INT8, and FP8 configurations at different input/weight toggle rates.

TABLE I
PERFORMANCE COMPARISON WITH STATE-OF-THE-ART DIGITAL SRAM-BASED CIM MACROS.

	VLSI'22 [8]	JSSC'22 [9]	ESSCIRC'22 [27]	TCAS2'23 [21]	ESSCIRC'23 [22]	TCAS1'24 [23]	This work
Technology	12nm	28nm	28nm	40nm	28nm	55nm	40nm
Array Size	8Kb	12Kb	16Kb	128Kb	4Kb	4Kb	36Kb
Supply Voltage	0.72V	0.6-1.0V	0.5-0.9V	0.9-1.2V	0.55-1.2V	0.7-1.2V	0.7-1.2V
INT Precision	INT4, INT8	INT8, INT16	NS	INT1-INT16	NS	INT1-INT8	INT4, INT8
FP Precision	NS	BF16, FP32	FP8-FP32	NS	BF8, FP8	NS	FP8
Peak Throughput (GOPS or GFLOPS)	1343 (INT4) 336* (INT8)	1350 (INT8) 340 (INT16) 1080 (BF16) 140 (FP32)	2.54 (FP8) 0.21 (FP32)	9600 (INT1) 600 (INT4) 150 (INT8) 37.5 (INT16)	2420 (BF8) 670 (FP8)	819.2 (INT1) 51.2 (INT4) 12.8 (INT8)	7123.48 (INT4) 1780.87 (INT8) 1187.25 (FP8)
Peak Energy Efficiency (TOPS/W or TFLOPS/W)	121 (INT4) 30.3* (INT8)	57.8 (INT8) 14.4 (INT16) 46.2 (BF16) 5.9 (FP32)	1.644 (FP8) 0.099 (FP32)	497 (INT1) 31.06 (INT4) 7.77 (INT8) 1.94 (INT16)	66.6 (BF8) 12.1 (FP8)	2238 (INT1) 44.82 (INT4) 14.2 (INT8)	367.45 (INT4) 77.25 (INT8) 23.14 (FP8)
Area Efficiency (TOPS/mm² or TFLOPS/mm²)	41.58 (INT4) 10.4* (INT8)	1.43 (INT8) 0.36 (INT16) 1.15 (BF16) 0.15 (FP32)	0.06 (FP8) 0.005 (FP32)	23.41 (INT1) 1.46 (INT4) 0.37 (INT8) 0.09 (INT16)	1.55 (BF8) 0.43 (FP8)	13.21 (INT1) 0.83 (INT4) 0.21 (INT8)	1.86 (INT4) 0.46 (INT8) 0.31 (FP8)

NS: Not supported, *: Estimated.

macro achieves remarkable performance in terms of throughput, energy efficiency, and area efficiency compared to existing works, while offering the flexibility of supporting both integer and floating-point MAC operations within a single architecture. Compared to the SOTA works [8]/ [9]/ [22] achieving the highest INT4/INT8/FP8 throughput, the proposed macro offers improvements of 5.3×/1.32×/1.77×, respectively. This is due to the parallel-input approach, which enables fast parallel processing. In addition, our macro offers 3.04×/1.34×/1.391× savings in energy efficiency for INT4/INT8/FP8 compared to [8]/ [9]/ [22], the highest energy efficient SOTA works. Our macro's energy efficiency is a result of using power gating which disables unused components to conserve energy when operating in a specific mode. Moreover, the proposed macro achieves comparable area efficiency values to SOTA designs. Particularly, it offers 55.38% and 54.35% increase in area efficiency compared to [23] for INT4 and INT8, respectively, and 80.65% compared to [27] for FP8. In addition, when scaled to the same array size and technology, the macro achieves area improvements of 1.5× and 7.46× compared to [9] and [22], respectively. Note that our comparison focuses on digital designs that support similar data formats to our macro. Other works, such as [20] and [28], were not included in the table to ensure a fair comparison with designs that match our data format specifications. The area breakdown of our design is summarized as follows: majority of the area (∼59%) is occupied by the multipliers, followed by the adder trees (∼19%). The product aligner accounts for 10%, the mantissa shifters 9%, and the SRAM bit cells less than 1%. The remaining 3% is allocated to the peripherals, NRU, and PSUM. The low area efficiency is due to the parallel processing of multi-bit inputs, which is the case in other works [22], [29].

IV. CONCLUSION

In this work, we proposed a reconfigurable digital SRAM-based compute-in-memory macro to accelerate both integer and floating-point MAC operations within a single architecture, specifically for INT4, INT8 and FP8 (E4M3) configurations. Our design utilizes a mantissa parallel-alignment method in FP mode and a parallel-input approach to enhance performance. The proposed macro is implemented in 40nm CMOS technology. The evaluation results demonstrate significant improvements over state-of-the-art digital SRAM-based CIM macros, achieving peak throughputs of 7123.48 GOPS in INT4 mode and 1187.25 GFLOPS in FP8 mode, and peak energy efficiencies of 367.45 TOPS/W and 23.14 TFLOPS/W, respectively. This makes the proposed macro a highly efficient and flexible solution for performing computations in various DNN models, particularly those involving mixed-precision computations. Future work includes evaluating the macro in real-world applications, such as NN and LLM inference with INT/FP mixed-precision models.

ACKNOWLEDGEMENT

This work has been partially supported by King Abdullah University of Science and Technology CRG program under grant number: URF/1/4704-01-01

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:31 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Goel, C. Tung, Y.-H. Lu, and G. K. Thiruvathukal, "A survey of methods for low-power deep learning and computer vision," in *2020 IEEE 6th World Forum on Internet of Things (WF-IoT)*. IEEE, 2020, pp. 1–6.

[2] Z.-Y. Dou, X. Wang, S. Shi, and Z. Tu, "Exploiting deep representations for natural language processing," *Neurocomputing*, vol. 386, pp. 1–7, 2020.

[3] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, "A survey of deep learning techniques for autonomous driving," *Journal of field robotics*, vol. 37, no. 3, pp. 362–386, 2020.

[4] P. Khan, M. F. Kader, S. R. Islam, A. B. Rahman, M. S. Kamal, M. U. Toha, and K.-S. Kwak, "Machine learning and deep learning approaches for brain disease diagnosis: principles and recent advances," *Ieee Access*, vol. 9, pp. 37 622–37 655, 2021.

[5] Y. Ma, Y. Du, L. Du, J. Lin, and Z. Wang, "In-memory computing: The next-generation ai computing paradigm," in *Proceedings of the 2020 on Great Lakes Symposium on VLSI*, 2020, pp. 265–270.

[6] J. Bazzi, R. Jamil, D. ElHajj, R. Kanj, M. E. Fouda, and A. Eltawil, "Reconfigurable precision sram-based analog in-memory-compute macro design," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2024, pp. 1–5.

[7] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable sram-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, 2021.

[8] C.-F. Lee, C.-H. Lu, C.-E. Lee, H. Mori, H. Fujiwara, Y.-C. Shih, T.-L. Chou, Y.-D. Chih, and T.-Y. J. Chang, "A 12nm 121-tops/w 41.6-tops/mm2 all digital full precision sram-based compute-in-memory with configurable bit-width for ai edge applications," in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2022, pp. 24–25.

[9] F. Tu, Y. Wang, Z. Wu, L. Liang, Y. Ding, B. Kim, L. Liu, S. Wei, Y. Xie, and S. Yin, "Redcim: Reconfigurable digital computing-in-memory processor with unified fp/int pipeline for cloud ai acceleration," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, 2022.

[10] J.-M. Hung, Y.-H. Huang, S.-P. Huang, F.-C. Chang, T.-H. Wen, C.-I. Su, W.-S. Khwa, C.-C. Lo, R.-S. Liu, C.-C. Hsieh *et al.*, "An 8-mb dc-current-free binary-to-8b precision reram nonvolatile computing-in-memory macro using time-space-readout with 1286.4-21.6 tops/w for edge-ai devices," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022.

[11] C.-X. Xue, T.-Y. Huang, J.-S. Liu, T.-W. Chang, H.-Y. Kao, J.-H. Wang, T.-W. Liu, S.-Y. Wei, S.-P. Huang, W.-C. Wei *et al.*, "15.4 a 22nm 2mb reram compute-in-memory macro with 121-28tops/w for multibit mac computing for tiny ai edge devices," in *2020 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2020, pp. 244–246.

[12] C.-X. Xue, J.-M. Hung, H.-Y. Kao, Y.-H. Huang, S.-P. Huang, F.-C. Chang, P. Chen, T.-W. Liu, C.-J. Jhang, C.-I. Su *et al.*, "16.1 a 22nm 4mb 8b-precision reram computing-in-memory macro with 11.91 to 195.7 tops/w for tiny ai edge devices," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 245–247.

[13] S. Xie, C. Ni, A. Sayal, P. Jain, F. Hamzaoglu, and J. P. Kulkarni, "16.2 edram-cim: Compute-in-memory design with reconfigurable embedded-dynamic-memory array realizing adaptive data converters and charge-domain computing," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 248–250.

[14] Y.-C. Chiu, C.-S. Yang, S.-H. Teng, H.-Y. Huang, F.-C. Chang, Y. Wu, Y.-A. Chien, F.-L. Hsieh, C.-Y. Li, G.-Y. Lin *et al.*, "A 22nm 4mb stt-mram data-encrypted near-memory computation macro with a 192gb/s read-and-decryption bandwidth and 25.1-55.1 tops/w 8b mac for ai operations," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 178–180.

[15] L. Liu, L. Tan, J. Gan, B. Pan, J. Zhou, and Z. Li, "Mdcim: Mram-based digital computing-in-memory macro for floating-point computation with high energy efficiency and low area overhead," *Applied Sciences*, vol. 13, no. 21, p. 11914, 2023.

[16] H. Wang, R. Liu, R. Dorrance, D. Dasalukunte, D. Lake, and B. Carlton, "A charge domain sram compute-in-memory macro with c-2c ladder-based 8-bit mac unit in 22-nm finfet process for edge inference," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, 2023.

[17] H. Zhang, S. He, X. Lu, X. Guo, S. Wang, Y. Du, and L. Du, "Ssm-cim: An efficient cim macro featuring single-step multi-bit mac computation for cnn edge inference," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2023.

[18] H. Jia, M. Ozatay, Y. Tang, H. Valavi, R. Pathak, J. Lee, and N. Verma, "Scalable and programmable neural network inference accelerator based on in-memory computing," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, 2021.

[19] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, "A programmable heterogeneous microprocessor based on bit-scalable in-memory computing," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 9, pp. 2609–2621, 2020.

[20] Y. Mai, M. Wang, C. Zhang, B. Zhong, and Z. Yu, "A 1.97 tflops/w configurable sram-based floating-point computation-in-memory macro for energy-efficient ai chips," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–5.

[21] C. Zhang, M. Wang, Y. Mai, C. Tang, and Z. Yu, "A high-density and reconfigurable sram-based digital compute-in-memory macro for low-power ai chips," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 9, pp. 3589–3593, 2023.

[22] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "Fp-imc: A 28nm all-digital configurable floating-point in-memory computing macro," in *ESSCIRC 2023-IEEE 49th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2023, pp. 405–408.

[23] H. You, W. Li, D. Shang, Y. Zhou, and S. Qiao, "A 1–8b reconfigurable digital sram compute-in-memory macro for processing neural networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.

[24] B. Yan, J.-L. Hsu, P.-C. Yu, C.-C. Lee, Y. Zhang, W. Yue, G. Mei, Y. Yang, Y. Yang, H. Li *et al.*, "A 1.041-mb/mm 2 27.38-tops/w signed-int8 dynamic-logic-based adc-less sram compute-in-memory macro in 28nm with reconfigurable bitwise operation for ai and embedded applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 188–190.

[25] Y.-D. Chih, P.-H. Lee, H. Fujiwara, Y.-C. Shih, C.-F. Lee, R. Naous, Y.-L. Chen, C.-P. Lo, C.-H. Lu, H. Mori *et al.*, "16.4 an 89tops/w and 16.3 tops/mm 2 all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 252–254.

[26] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf *et al.*, "Microscaling data formats for deep learning," *arXiv preprint arXiv:2310.10537*, 2023.

[27] S. Jeong, J. Park, and D. Jeon, "A 28nm 1.644 tflops/w floating-point computation sram macro with variable precision for deep neural network inference and training," in *ESSCIRC 2022-IEEE 48th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2022, pp. 145–148.

[28] Y. Mai, M. Wang, B. Zhong, C. Zhang, Y. Zhang, and Z. Yu, "An energy-efficient floating-point compute sram with pipelined in-memory bit-parallel exponent and bitwise mantissa processing," *Electronics Letters*, vol. 59, no. 14, p. e12885, 2023.

[29] C.-F. Lee, C.-H. Lu, C.-E. Lee, H. Mori, H. Fujiwara, Y.-C. Shih, T.-L. Chou, Y.-D. Chih, and T.-Y. J. Chang, "A 12nm 121-tops/w 41.6-tops/mm2 all digital full precision sram-based compute-in-memory with configurable bit-width for ai edge applications," in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2022, pp. 24–25.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:31 UTC from IEEE Xplore. Restrictions apply.